Individual Reflection

Jonathan Immanuel / 14487692

Worst Case and Input Triggers

The worst-case running time of my Median of Medians implementation is linear rather than quadratic. After dividing the active interval into groups of five, the algorithm selects the median of those group medians. Apart from incomplete groups and rounding, this guarantees that roughly three-tenths of the values can be removed from consideration. Therefore, the largest remaining strict partition contains at most about seven-tenths of the values. The algorithm also performs a smaller selection on approximately one-fifth of the input to construct the pivot. This gives the recurrence:

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + O(n) = O(n)$$

Hardest Design Decision

The hardest design decision was how to collect the group medians without creating a separate vector. I sorted each group and swapped its middle value into a prefix of the original vector. The recursive call then selects a pivot from that prefix. This avoided a separate allocation, but it made the indexing harder because `k` remained an absolute vector position while the active interval became smaller. I needed tests for incomplete groups and ranks near the interval boundaries to confirm that the indexing stayed correct.

Three-way partitioning was another important decision. A two-way partition can repeatedly process values equal to the pivot. With three regions, values smaller than, equal to and greater than the pivot are separated in one partitioning step. If the requested rank falls inside the equal region, the algorithm can finish immediately. This was particularly important for the random-duplicate and all-equal benchmarks.

What I Would Do Differently

If I started the project again, I would define the full interface contract and test matrix before implementing the recursive pivot step. I would also add counters for comparisons, swaps, partition passes and the total number of elements scanned from the beginning. Timing showed us which implementation was faster, but it could not completely explain why the difference occurred.

What I Now Understand

Before this project, I understood that Median of Medians had linear worst-case time, but I did not understand where that guarantee came from. I now understand that the pivot does not need to be the exact median. It only needs to guarantee that a fixed fraction of the active interval is removed during each recursive step.

The benchmark also made the difference between theoretical protection and practical speed clearer. Randomised Quickselect had a lower median running time in 324 of the 330 tested cases because selecting a random pivot required much less preparation. Median of Medians had a lower median time in six small cases, including three consecutive duplicate-heavy cases using the middle rank. This showed me

that the extra work required to construct a deterministic pivot can sometimes be recovered when that pivot produces a useful equal region. However, this did not make Median of Medians the faster algorithm overall.